

ui-connector deploy runbook (header auth)

2026-09-22 · @Someone

Build and push the header-auth image from Cloud Shell, deploy it with `kubectl apply -f deployment.yaml` from the bastion, and keep a one-file rollback.

Before you start

This release reads the token **only** from `Authorization: Bearer <token>`. The LWC can't set headers on a browser WebSocket, which is why `?token=` was adopted in July, so LWC connections will close with `1008 missing_token` until the client changes. Test with Postman and keep step 6 ready.

- **Cloud Shell, steps 1-3:** back up the current image, build, push. Cloud Shell has Docker.
- **Bastion, steps 4-6:** `kubectl apply`, verify, roll back. Every command uses long flags.
- **Files:** in Cloud Shell, `:` menu → Upload → `ui-connector.zip`, then `unzip ui-connector.zip`. The folder must hold `Dockerfile`, `main.py`, `utils.py`, `requirements.txt` and `connector/`.
- **Dockerfile:** use the one from the zip. Your current one copies only `main.py`, which crash-loops on `ModuleNotFoundError`, the same failure as in August. The new one fails the build instead, via its `IMPORT OK` check.
- **Tags:** push every build under a new tag (`v3-header-auth`, then `v4`, ...) and never reuse one. A new tag has no cached copy on any node, so `kubectl apply` pulls it fresh. The stale `:v2` in August came from overwriting a tag.
- **TST:** same steps with the TST project, repo, namespace and deployment names.

1. Cloud Shell: set variables

`IMG` must match the `image:` line in your `deployment.yaml`, without the `:tag` or `@sha256:...` on the end. If the session drops, run this block again.

```
gcloud config set project adtgcp-ent-dev-ccs-tlcm-fe10
gcloud auth configure-docker us-central1-docker.pkg.dev --quiet
export REG=us-central1-docker.pkg.dev/adtgcp-ent-dev-ccs-tlcm-fe10
export IMG=$REG/ui-connector/ui-connector
export TAG=v3-header-auth
```

2. Cloud Shell: back up the current image

First find what's running: on the bastion, `grep image: deployment.yaml` . Note the tag, or the first 8 characters after `sha256:` .

```
# recent pushes: DIGEST, TAGS, CREATE_TIME
gcloud artifacts docker images list $IMG --include-tags \
  --sort-by=~CREATE_TIME --limit=5

# the running one, by the 8 digest characters you noted
export D=3bc46f89      # yours
export OLD=$(gcloud artifacts docker images list $IMG \
  --filter="version~^sha256:$D" \
  --format='value[separator="@"](package,version)')
echo $OLD
```

`echo` must print one line ending in `@sha256:...` . If `deployment.yaml` uses a tag instead, run `export OLD=$IMG:<that tag>` .

```
# pull it, keep the whole image, extract the code
docker pull $OLD
mkdir -p ~/backup-current
docker save --output ~/backup-current/image.tar $OLD
docker create --name old-app $OLD
docker cp old-app:/app ~/backup-current/app
docker rm old-app
ls ~/backup-current/app

# name it in Artifact Registry so it can't get lost
gcloud artifacts docker tags add $OLD $IMG:backup-pre-header-auth

# optional: download the code to your computer
tar czf ~/app-backup.tgz -C ~/backup-current app
cloudshell download ~/app-backup.tgz
```

`ls` should show `main.py` and `requirements.txt` .

3. Cloud Shell: build and push

```
cd ~/ui-connector
ls connector      # must list: auth  routes  ws  app.py  config.py ...
docker build --progress=plain --tag $IMG:$TAG .
docker push $IMG:$TAG
```

- The build output must show `IMPORT OK`. If it doesn't, you're in the wrong folder or on the old Dockerfile.
- The push ends with `v3-header-auth: digest: sha256:...`. Note the first 8 characters after `sha256:` to check the pods against in step 5.

4. Bastion: deploy with kubectl apply

Keep a copy of the current file for rollback, point the `image:` line at the new tag, apply.

```
export NS=ui-connector
export DEP=adt-ui-connectorv2-1
export REG=us-central1-docker.pkg.dev/adtgcp-ent-dev-ccs-tlcm-fe10
export IMG=$REG/ui-connector/ui-connector
kubectl config set-context --current --namespace=$NS

cd <folder with deployment.yaml>
cp deployment.yaml deployment.yaml.bak
sed -i "s|image: .*|image: $IMG:v3-header-auth|" deployment.yaml
grep -n "image" deployment.yaml

kubectl apply -f deployment.yaml
kubectl rollout status deployment/$DEP --timeout=180s
kubectl get pods --selector=app=$DEP
```

- `grep` must show the new tag on the `image:` line; `sed` leaves `imagePullPolicy` alone.
- **Env vars:** no changes needed. The new code reads the same names (`WS_TOKEN_AUDIENCE`, `ALLOWED_SAS`, `REDISHOST` ...). Check with `kubectl set env deployment/$DEP --list`.

5. Bastion: verify

```
# after sha256: the IMAGE column must start with the 8 characters from step 3
export COLS='POD:.metadata.name,IMAGE:.status.containerStatuses[0].imageID'
kubectl get pods --selector=app=$DEP --output=custom-columns="$COLS"

# startup line: "- | ui-connector | Started"
kubectl logs deployment/$DEP --tail=30

# health endpoints
kubectl port-forward deployment/$DEP 8080:8080 &
curl -s localhost:8080/
curl -s localhost:8080/auth-health
kill %1
```

Expect "redis": "connected" from /, and "google_certs": "reachable" plus "audience_configured": true from /auth-health.

Postman, New → WebSocket:

Test	URL	Header	Expected
Header auth	wss://dev.agentassist.adt.com/ws?conversation_id=conv-test1234	Authorization: Bearer <fresh token>	HISTORY_BATCH frame
No header	same URL	none	close 1008 missing_token
Old style	same URL + &token=<token>	none	close 1008 missing_token

Then `kubectl logs deployment/$DEP --tail=50 | grep "conv-test1234 |"` should show the `Connected` and `Session closed` lines.

6. Bastion: roll back

`deployment.yaml.bak` still points at the old image, so rolling back is one apply.

```
kubectl apply -f deployment.yaml.bak
kubectl rollout status deployment/$DEP --timeout=180s
cp deployment.yaml.bak deployment.yaml
```

The old image is also in Artifact Registry as `backup-pre-header-auth`, and its code is in `~/backup-current/app` in Cloud Shell.